

CAHIER DES CHARGES — DUOPLAN v1.0

Mini-Agent d'Orchestration et de Négociation pour Vacances en Couple

Auteur :	Quentin Léger (BUT Informatique, Nancy)
Objectif IL :	Structured Outputs, SQLite, multi-agents, architecture propre sans UI complexe
Stack Cible :	Python 3.11+, Streamlit, Groq (Llama 3.3), Pydantic, SQLite

1. Présentation Générale & Objectifs

L'application **DuoPlan** a pour but de résoudre un problème quotidien récurrent lors des moments de pause ou des trajets : le choix d'une activité commune (film, restaurant, sortie) lorsque les deux partenaires expriment des envies divergentes.

Loin d'être un simple script jetable, ce projet sert de support d'apprentissage pratique pour le parcours **Ingénierie Logicielle (IL)** en vue des semestres 3 et 4 du BUT et des admissions en école d'ingénieurs (Télécom Nancy). L'accent est mis sur la robustesse du code, l'utilisation de formats de données stricts, et la persistance.

2. Spécifications Fonctionnelles

A. Collecte des Envies & Profils

L'interface graphique épurée (générée via Streamlit) doit proposer deux champs de saisie distincts : un pour Quentin, un pour sa copine. L'application doit également permettre de sélectionner le type d'activité recherché (Cinéma/Film, Restaurant, Visite/Activité).

B. Intelligence Artificielle & Négociation Multi-Agents

Le traitement de la requête s'articule autour d'une logique d'orchestration à deux niveaux :

- **L'Agent Négociateur** : Analyse les deux saisies textuelles brutes et formule trois propositions de compromis distinctes au format structuré.
- **L'Agent Arbitre** : Analyse les trois propositions issues du premier agent, évalue la balance du compromis via un score d'équité, et rédige une justification humoristique et personnalisée pour chaque membre du couple.

C. Persistance et Historique (Module SQL)

Afin de mettre en application les compétences en bases de données, chaque décision validée par le couple doit être consignée dans une base locale `SQLite`. Les données stockées incluent les envies de départ, la proposition retenue, la date et le score d'équité.

Indicateur d'Équité : L'application calcule un taux cumulé d'arbitrage $E = \Sigma (C_i) / N$ où C_i représente l'avantage accordé à l'un ou l'autre sur la session i , permettant à l'IA d'ajuster ses propositions futures pour éviter qu'une même personne ne cède systématiquement.

3. Architecture Technique & Robustesse (Cœur IL)

Garantie du format de données (Structured Outputs)

Afin de bannir les nettoyages fragiles de chaînes de caractères (type `split("```")`), le contrat d'interface avec l'API Groq est régi par un schéma de validation strict basé sur **Pydantic**. La structure attendue en retour de l'LLM est définie comme suit :

- `titre_proposition` (string) : Le nom de l'activité ou du film recommandé.
- `pourquoi_quentin` (string) : Argumentation ciblée pour Quentin.
- `pourquoi_copine` (string) : Argumentation ciblée pour sa copine.
- `note_compromis` (integer, de 1 à 10) : Note globale d'adéquation aux deux profils.

Zéro Graphisme (Focus Streamlit)

Le développement de l'interface utilisateur s'affranchit totalement des technologies front-end classiques (HTML/CSS). L'implémentation repose exclusivement sur les primitives de mise en page natives de Streamlit (`st.columns`, `st.chat_input`, `st.dataframe`), assurant un rendu propre, moderne et responsive, idéal pour un usage sur PC portable ou smartphone en déplacement.

4. Phases de Développement (Feuille de Route 10 Jours)

1. **Phase 1 (Train Aller)** : Initialisation du dépôt Git, configuration de l'environnement virtuel, script d'appel basique à Groq et mise en place de la première page Streamlit.
2. **Phase 2 (Breaks / Soirs)** : Implémentation du modèle Pydantic, sécurisation de la fonction `ask_nova_compromis` avec le paramètre `response_format`.
3. **Phase 3 (Train Retour)** : Création du schéma de table SQLite, écriture des fonctions d'insertion SQL et mise en place de l'affichage de l'historique des vacances.